

TUGAS GRAFIK DAN TEKNIK INTERAKTIF
TRANSFORMASI GEOMETRI CITRA DIGITAL



Disusun Oleh:

Hasta Putra Wildantara

24060124130119

PROGRAM STUDI INFORMATIKA
FAKULTAS SAINS DAN MATEMATIKA
UNIVERSITAS DIPONEGORO
2026

LAPORAN TUGAS DETEKSI TEPI ROBERTS

Implementasi Operator Roberts 2x2 pada Citra Kucing Menggunakan GNU Octave

1. Pendahuluan

Deteksi tepi merupakan salah satu proses penting dalam pengolahan citra digital karena digunakan untuk menemukan batas objek berdasarkan perubahan intensitas piksel. Pada tugas ini, deteksi tepi diterapkan pada citra kucing menggunakan operator Roberts yang memakai kernel berukuran 2x2.

Tujuan program adalah membaca citra, mengubahnya ke grayscale apabila masih berupa citra RGB, menghitung respons tepi dengan dua kernel Roberts, kemudian melakukan thresholding untuk menghasilkan citra biner yang menampilkan bagian tepi objek.

2. Dasar Teori Operator Roberts

Operator Roberts bekerja dengan membandingkan perbedaan intensitas secara diagonal pada piksel-piksel yang berdekatan. Karena menggunakan kernel 2x2, operator ini sederhana dan cocok untuk menunjukkan konsep dasar konvolusi serta deteksi perubahan intensitas lokal.

Dua kernel yang digunakan adalah R+ untuk arah diagonal satu dan R- untuk arah diagonal lainnya. Besar tepi dihitung dari jumlah nilai absolut kedua respons tersebut, yaitu $G = |R+| + |R-|$. Nilai magnitude kemudian dibandingkan dengan threshold T. Jika magnitude lebih besar atau sama dengan T, piksel dianggap sebagai tepi dan diberi nilai 255; jika tidak, piksel diberi nilai 0.

3. Alur Program

1. Program utama membaca file citra kucing.jpeg menggunakan fungsi `imread`.
2. Jika citra memiliki tiga kanal warna, citra dikonversi ke grayscale dengan `rgb2gray`.
3. Fungsi `roberts` membentuk dua kernel 2x2, yaitu kernel R+ dan R-.
4. Fungsi konvolusi 2x2 menghitung respons setiap kernel secara manual pada seluruh area citra.
5. Magnitude tepi dihitung dengan menjumlahkan nilai absolut respons R+ dan R-.
6. Fungsi thresholding mengubah magnitude menjadi citra biner berdasarkan nilai ambang T.
7. Program menguji tiga nilai threshold, yaitu 64, 128, dan 192, lalu menyimpan hasilnya dalam format PNG.

4. Kode Program Octave

Kode program terdiri atas empat file utama. File `konvolusi2x2.m` melakukan operasi konvolusi manual, `roberts.m` menghitung respons operator Roberts, `thresholding.m` melakukan binarisasi, dan `main_roberts.m` menjadi script utama untuk menjalankan percobaan.

konvolusi2x2.m

```
% konvolusi2x2.m
% Konvolusi manual untuk kernel 2x2 (digunakan oleh operator Roberts)
function hasil = konvolusi2x2(citra, kernel)
    [H, W] = size(citra);
    citra_d = double(citra);
    hasil = zeros(H, W);

    % Iterasi hanya sampai H-1 dan W-1 karena kernel 2x2
    for y = 1:H-1
        for x = 1:W-1
            % Ambil region 2x2
            region = citra_d(y:y+1, x:x+1);
            % Jumlahkan hasil perkalian element-wise
            hasil(y, x) = sum(sum(region .* kernel));
        end
    end
end
```

roberts.m

```
% roberts.m
% =====
% DETEKSI TEPI - OPERATOR ROBERTS (kernel 2x2)
% =====
%
% Rumus:
%  $R+(x,y) = f(x+1, y+1) - f(x, y)$  → diagonal /
%  $R-(x,y) = f(x, y+1) - f(x+1, y)$  → diagonal \
%
% Kernel:
%  $R+ = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$        $R- = \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix}$ 
%
% Magnitude Tepi:
%  $G[f(x,y)] = |R+| + |R-|$ 
%
% Kemudian dilakukan thresholding dengan nilai ambang T.
%
% PARAMETER:
% citra - matriks grayscale uint8
% T - nilai ambang (0-255), default 128
%
% OUTPUT:
% hasil - citra biner hasil deteksi tepi (uint8)
% magnitude - peta kekuatan tepi sebelum thresholding
% Rp - respons kernel R+ (diagonal /)
% Rm - respons kernel R- (diagonal \)
%
% Contoh:
% hasil = roberts(citra, 128);
% [hasil, mag] = roberts(citra, 100);
% =====

function [hasil, magnitude, Rp, Rm] = roberts(citra, T)
    if nargin < 2
        T = 128; % nilai ambang default
    end

    % — Kernel Roberts 2x2 —————
    % R+ mendeteksi tepi diagonal kiri-bawah ke kanan-atas (/)
    kernel_Rp = [ 1 0; ...
```

```

        0 -1];

% R- mendeteksi tepi diagonal kanan-bawah ke kiri-atas (\)
kernel_Rm = [ 0  1; ...
             -1  0];

% — Konvolusi citra dengan kedua kernel —————
Rp = konvolusi2x2(citra, kernel_Rp); % respons diagonal /
Rm = konvolusi2x2(citra, kernel_Rm); % respons diagonal \

% — Hitung magnitude tepi: G = |R+| + |R-| —————
magnitude = abs(Rp) + abs(Rm);

% — Thresholding untuk menghasilkan citra biner —————
hasil = thresholding(magnitude, T);
end

```

thresholding.m

```

% thresholding.m
% Binarisasi citra berdasarkan nilai ambang T
% Piksel >= T → 255 (putih/tepi), piksel < T → 0 (hitam)
function hasil = thresholding(citra, T)
    [H, W] = size(citra);
    hasil = zeros(H, W, 'uint8');

    for y = 1:H
        for x = 1:W
            if citra(y, x) >= T
                hasil(y, x) = 255;
            else
                hasil(y, x) = 0;
            end
        end
    end
end
end

```

main_roberts.m

```

% main_roberts.m
% =====
% DETEKSI TEPI OPERATOR ROBERTS - SCRIPT UTAMA
% Mata Kuliah: Pengolahan Citra Digital (GKV - IF 2020)
% =====
%
% File yang diperlukan di folder yang sama:
% konvolusi2x2.m thresholding.m roberts.m kucing.jpeg
%
% Cara menjalankan di Octave/MATLAB:
% >> main_roberts
% =====

clc; clear; close all;

fprintf('=====\n');
fprintf(' DETEKSI TEPI - OPERATOR ROBERTS (2x2)\n');
fprintf('=====\n\n');

%% — BACA CITRA —————
citra_raw = imread('kucing.jpeg');

% Konversi ke grayscale jika RGB
if size(citra_raw, 3) == 3
    citra = rgb2gray(citra_raw);
else
    citra = citra_raw;
end

[H, W] = size(citra);
fprintf('[INFO] File   : kucing.jpeg\n');
fprintf('[INFO] Ukuran : %d x %d piksel\n\n', W, H);

```

```

%% — KERNEL ROBERTS —————
% Ditampilkan agar sesuai materi kuliah
Rp_kernel = [ 1 0; 0 -1]; % diagonal /
Rm_kernel = [ 0 1; -1 0]; % diagonal \

fprintf('Kernel R+ (diagonal /):\n');
disp(Rp_kernel);
fprintf('Kernel R- (diagonal \):\n');
disp(Rm_kernel);

%% — DETEKSI TEPI DENGAN 3 NILAI AMBANG —————
% Percobaan dengan T = 64, 128, 192 untuk melihat perbedaan
ambang_list = [64, 128, 192];
label_list = {'T = 64 (sensitif)', ...
              'T = 128 (standar)', ...
              'T = 192 (ketat)'};

for k = 1:3
    T = ambang_list(k);
    fprintf('%d] Memproses dengan ambang T = %d ...\n', k, T);

    [hasil, magnitude, Rp, Rm] = roberts(citra, T);

    % Simpan hasil
    nama_file = sprintf('hasil_roberts_T%d.png', T);
    imwrite(hasil, nama_file);
    fprintf('    Disimpan -> %s\n', nama_file);

    % Tampilkan
    figure(k);
    subplot(2, 3, 1);
    imshow(citra);
    title('Citra Asal (Grayscale)', 'FontSize', 10, 'FontWeight', 'bold');

    subplot(2, 3, 2);
    imshow(uint8(abs(Rp) / max(abs(Rp(:))) * 255));
    title('Respons R+ (diagonal /)', 'FontSize', 10);

    subplot(2, 3, 3);
    imshow(uint8(abs(Rm) / max(abs(Rm(:))) * 255));
    title('Respons R- (diagonal \)', 'FontSize', 10);

    subplot(2, 3, [4 5]);
    imshow(uint8(magnitude / max(magnitude(:)) * 255));
    title('Magnitude Tepi (|R+| + |R-|)', 'FontSize', 10);

    subplot(2, 3, 6);
    imshow(hasil);
    title(label_list{k}, 'FontSize', 10, 'FontWeight', 'bold');

    % Judul figure – cetak ke konsol (Octave tidak punya sgtitle)
    fprintf('    Figure %d: Roberts Edge Detection - T = %d\n', k, T);
end

%% — PERBANDINGAN KETIGA AMBANG DALAM SATU FIGURE —————
figure(4);
subplot(1, 4, 1);
imshow(citra);
title('Asal', 'FontSize', 11, 'FontWeight', 'bold');

for k = 1:3
    T = ambang_list(k);
    [hasil_cmp, ~, ~, ~] = roberts(citra, T);
    subplot(1, 4, k+1);
    imshow(hasil_cmp);
    title(sprintf('T = %d', T), 'FontSize', 11);
end
fprintf('Figure 4: Perbandingan Nilai Ambang - Roberts\n');

fprintf('\n=====');
fprintf('    Selesai! Hasil disimpan sebagai PNG.\n');
fprintf('=====');

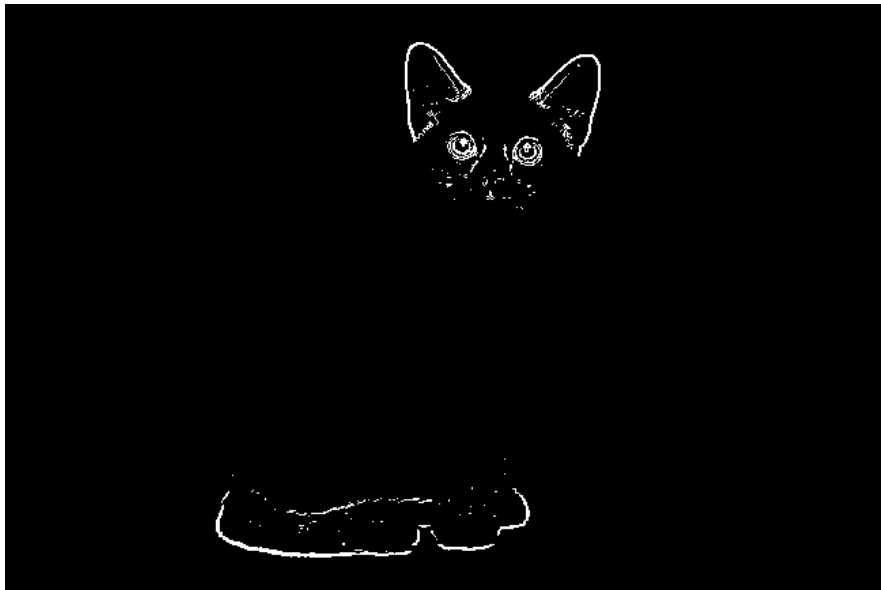
```

5. Hasil Pengujian

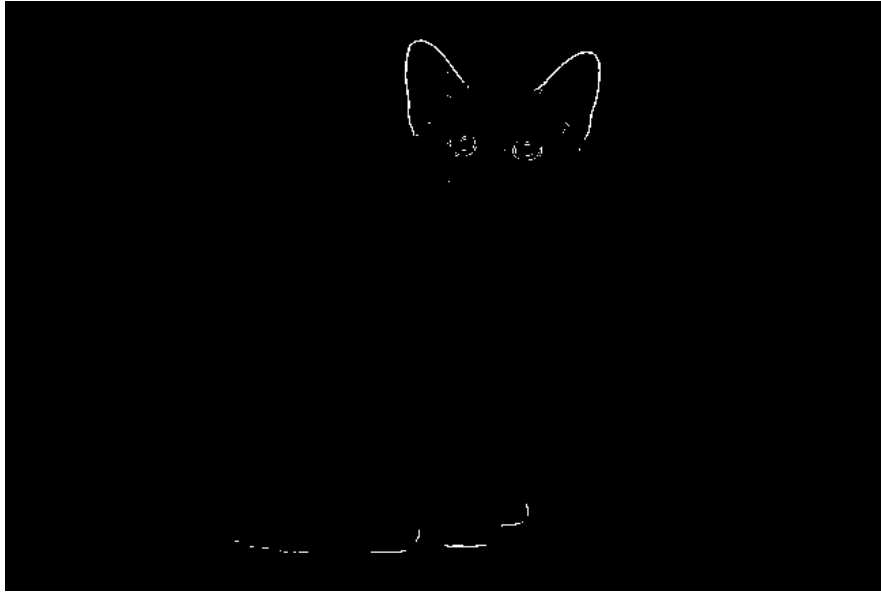
Citra masukan yang digunakan adalah gambar kucing berukuran 650 x 433 piksel. Program dijalankan dengan tiga nilai threshold untuk membandingkan pengaruh nilai ambang terhadap jumlah tepi yang terdeteksi.



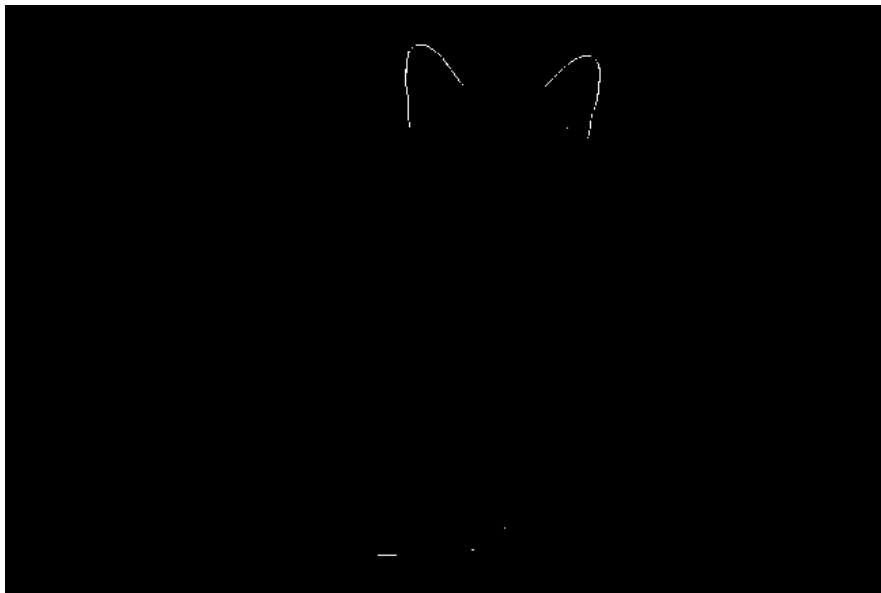
Gambar 1. Citra kucing sebagai data masukan.



Gambar 2. Hasil deteksi tepi Roberts dengan threshold $T = 64$.



Gambar 3. Hasil deteksi tepi Roberts dengan threshold $T = 128$.



Gambar 4. Hasil deteksi tepi Roberts dengan threshold $T = 192$.

Tabel Perbandingan Hasil

No.	Threshold	Pengamatan
1	$T = 64$	Ambang rendah menghasilkan tepi paling banyak. Detail pada telinga, mata, wajah, dan bagian bawah tubuh kucing masih cukup terlihat.
2	$T = 128$	Jumlah tepi berkurang. Garis utama pada telinga dan beberapa bagian wajah masih tampak, sedangkan detail kecil mulai hilang.

3	T = 192	Ambang tinggi hanya mempertahankan tepi yang paling kuat. Hasil menjadi lebih bersih tetapi banyak detail objek hilang.
---	---------	---

6. Pembahasan

Berdasarkan hasil percobaan, nilai threshold sangat memengaruhi banyaknya piksel yang dikategorikan sebagai tepi. Pada $T = 64$, program lebih sensitif sehingga lebih banyak detail kecil pada citra kucing yang ikut terdeteksi. Pada $T = 128$, hasil menjadi lebih selektif dan menampilkan tepi utama dengan detail yang lebih sedikit. Pada $T = 192$, hanya perubahan intensitas yang kuat yang dipertahankan sehingga jumlah garis tepi menjadi paling sedikit.

Dengan demikian, semakin besar nilai threshold, semakin sedikit tepi yang terdeteksi. Nilai threshold rendah cocok ketika detail tepi yang halus ingin dipertahankan, sedangkan threshold tinggi cocok ketika hasil yang diinginkan lebih bersih dan hanya menampilkan batas objek yang kuat.

7. Kesimpulan

- Operator Roberts dapat digunakan untuk mendeteksi tepi citra dengan dua kernel konvolusi berukuran 2×2 .
- Magnitude tepi diperoleh dari penjumlahan nilai absolut respons dua arah diagonal.
- Thresholding mengubah magnitude menjadi citra biner sehingga area tepi ditampilkan sebagai piksel putih.
- Percobaan pada citra kucing menunjukkan bahwa semakin besar nilai threshold, semakin sedikit tepi yang muncul pada hasil akhir.